

Nemotron-Labs-Diffusion

A Tri-Mode Language Model Unifying Autoregressive, Diffusion, and Self-Speculation Decoding

AR을 버리지 않고, diffusion을 병렬 초안과 생성 모드로 재 사용한다

01 AUTOREGRESSIVE

causal attention

02 DIFFUSION

block-bidirectional

03 SELF-SPECULATION

draft + verify

이 논문의 한 문장 결론

AR과 diffusion은 경쟁 관계가 아니라,
서로의 약점을 보완하는 두 개의 '동작 모드'다.

Q1 둘을 합칠 수 있는가? → 공동 loss와 attention 전환으로 통합

Q2 MTP보다 빠를 수 있는가? → diffusion draft + AR verify로 높은 수용 길이

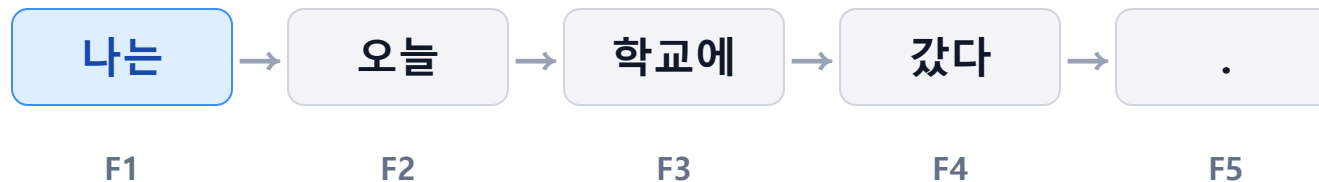
Q3 diffusion의 상한은 충분한가? → SOL 기준 실용 구현보다 큰 병렬성 여지

출발점: AR은 한 토큰씩 기다린다

1

token / forward

다음 토큰은 앞 토큰이
확정돼야 계산할 수 있다



저동시성에서는 메모리 대역폭이 병목

가중치를 매번 읽지만 한 번에 토큰 하나만 확정 → GPU 계산 자원이 충분히 채워지지 않음

텍스트 diffusion은 '가우시안 노이즈'가 아니다

IMAGE

Continuous diffusion

실수 latent에 Gaussian noise를 더하고
연속값을 조금씩 복원

$$\mathbf{z}_t = \sqrt{\alpha_t} \mathbf{z}_0 + \sqrt{(1-\alpha_t)} \boldsymbol{\varepsilon}$$

0.13 → 0.18 → 0.25 → 0.37

TEXT

Masked discrete diffusion

이산 token ID를 [MASK] 상태로 바꾸고
원래 범주를 반복적으로 분류

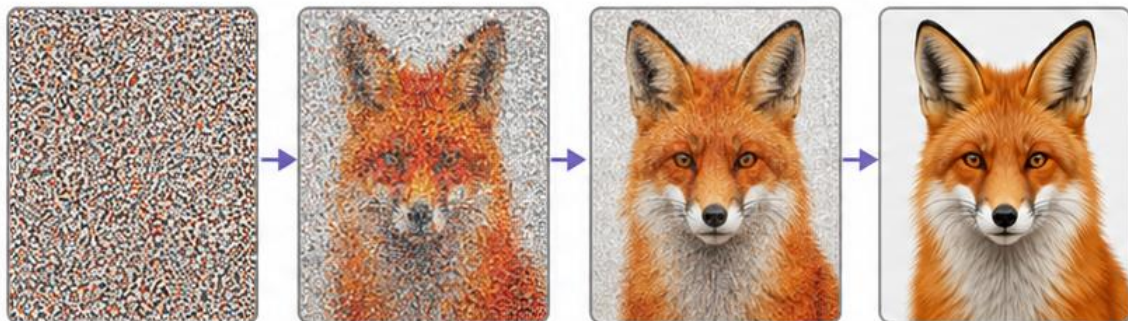
[MASK] → 'Seoul'

BERT식 복원을 생성 루프로 확장

공통점은 '오염 → 반복 복원'이고, 오염의 형태는 완전히 다르다.

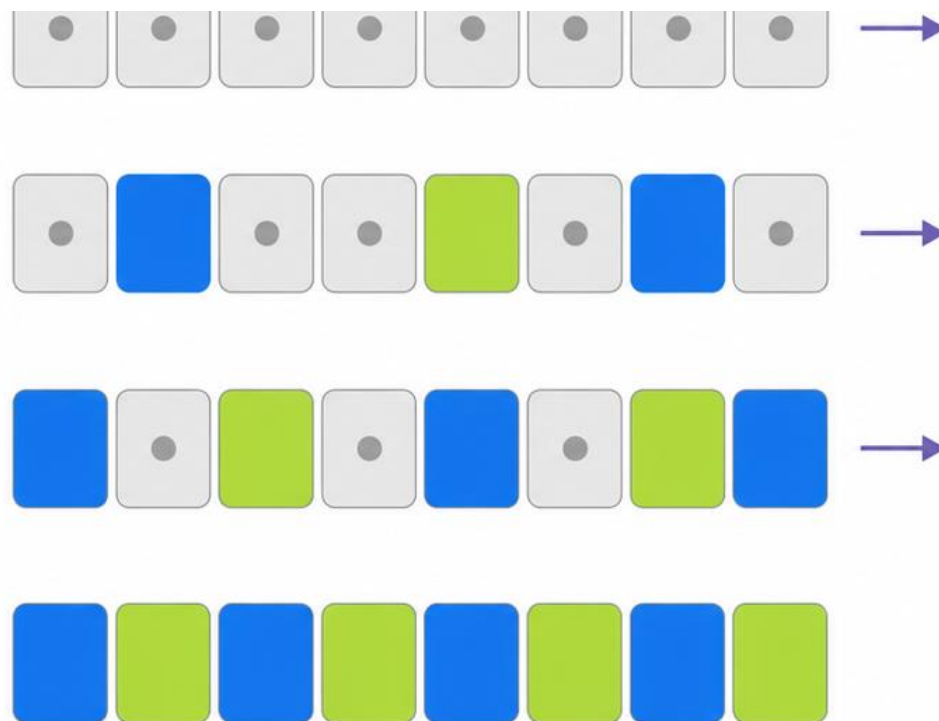
같은 diffusion이지만 '오염시키는 대상'이 다르다

IMAGE / CONTINUOUS



Gaussian noise: 연속적인 pixel/latent 값을 조금씩 복원

TEXT / MASKED DISCRETE



Masked discrete: [MASK] 위치의 vocabulary 범주를 다시 선택

생성 순서가 토큰 위치 순서와 분리된다

AR

위치 순서 = 확정 순서

나는	[]	[]	[]	[]
나는	오늘	[]	[]	[]
나는	오늘	학교에	갔다	.

DIFFUSION

위치는 고정, 확정 순서는 자유

[MASK]	[MASK]	[MASK]	[MASK]	[MASK]
[MASK]	오늘	[MASK]	[MASK]	.
나는	오늘	[MASK]	갔다	.
나는	오늘	학교에	갔다	.

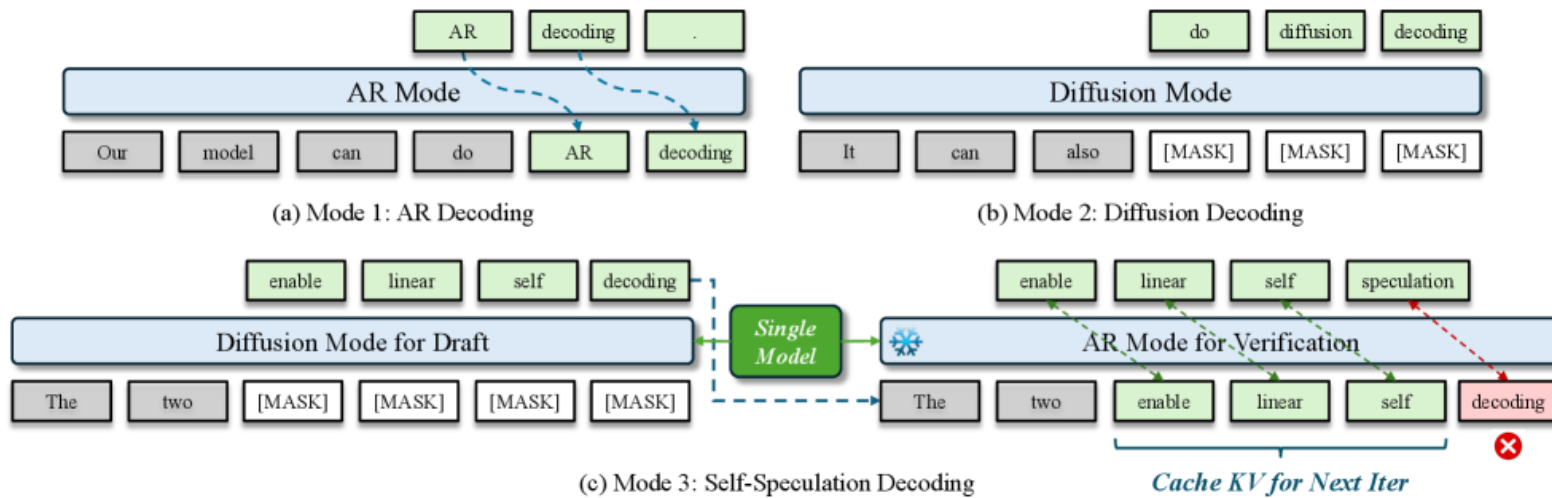
한 forward의 새 예측끼리 즉시 소통하는 것은 아니다. 일부를 확정 한 뒤 다음 forward에서 문맥이 된다.

블록 안은 양방향, 블록 사이는 causal



블록 생성: $1 \rightarrow 2 \rightarrow 3$ | 블록 내부 복원: confidence가 높은 위치부터 | 이전 블록 KV cache 재사용

백본은 하나, attention만 바꾼다



동일한 θ

causal mask
→ AR

block-bidirectional
→ Diffusion

두 mask를 연속 사용
→ Self-speculation

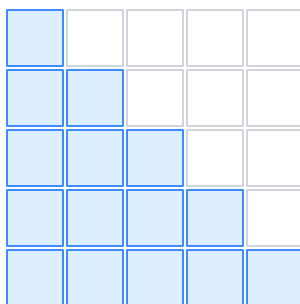
한 Transformer, 세 가지 attention schedule

Shared checkpoint θ

embedding · Transformer layers · LM head



MODE 1 · AR

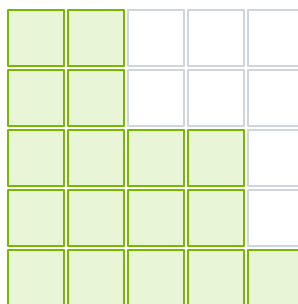


causal mask

한 token씩 확정



MODE 2 · DIFFUSION

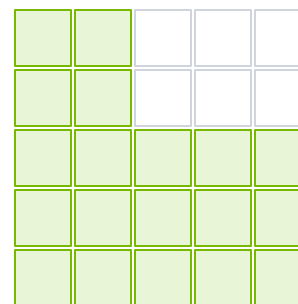


block-bidirectional

한 block 안 여러 위치 확정



MODE 3 · SELF-SPEC



draft mask → causal verify

같은 θ 를 두 schedule로 연속 실행

바뀌는 것: attention mask + decoding loop | 유지되는 것: 모델 가중치 θ

학습 목표는 AR + diffusion

AR OBJECTIVE

$$\mathcal{L}_{\text{AR}}(\theta) = \mathbb{E}_{\mathbf{x}} [- \sum_i \log p_{\theta}(\mathbf{x}_i \mid \mathbf{x}_{<i})]$$

깨끗한 문장에서 다음 토큰 예측 → 왼쪽에서 오른쪽이라는 언어 prior 유지

DIFFUSION OBJECTIVE

$$\mathcal{L}_{\text{diff}}(\theta) = \mathbb{E}_{\mathbf{t}, \mathbf{q}} [- 1/t \cdot \sum_{\beta} \log p_{\theta}(\mathbf{x}^{\beta} \mid \mathbf{x}_t^{\beta}, \mathbf{x}_{<^{\beta}})]$$

clean prefix에 조건화해 현재 noisy block을 복원 → 미래 토큰을 함께 고려

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{AR}}(\theta) + 0.3 \cdot \mathcal{L}_{\text{diff}}(\theta)$$

학습 MASK와 추론 sampler는 다르다

TRAINING

$t \sim \text{Uniform}[0,1]$

noise level에 비례해 현재 블록의 무작위 위치를 MASK 처리

t=0.2	나는	[M]	학교 에	갔다	.
t=0.5	[M]	오늘	[M]	갔다	[M]
t=0.8	[M]	[M]	[M]	갔다	[M]

INFERENCE

Sampler = acceptance policy

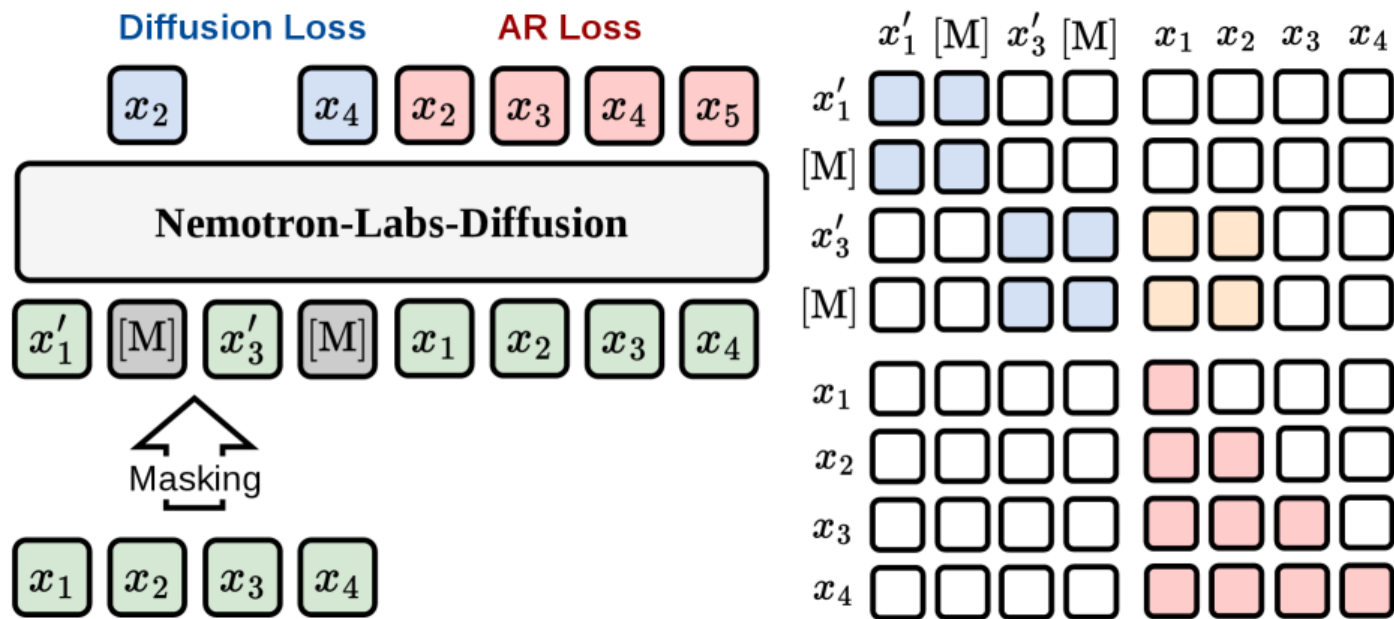
현재 MASK 중 어떤 top-1 예측을 지금 확정할지 선택

다음 MASK 집합 = 현재 MASK

– 이번 단계에서 확정된 위치

sampler가 반복 횟수를 직접 출력하지는 않는다

Dual-stream⁰이 두 loss를 한 번에 계산한다



Noisy stream

- 현재 block 내부 bidirectional
- 이전 clean prefix 참조
- 그 외 attention 경로 차단
- diffusion loss 계산

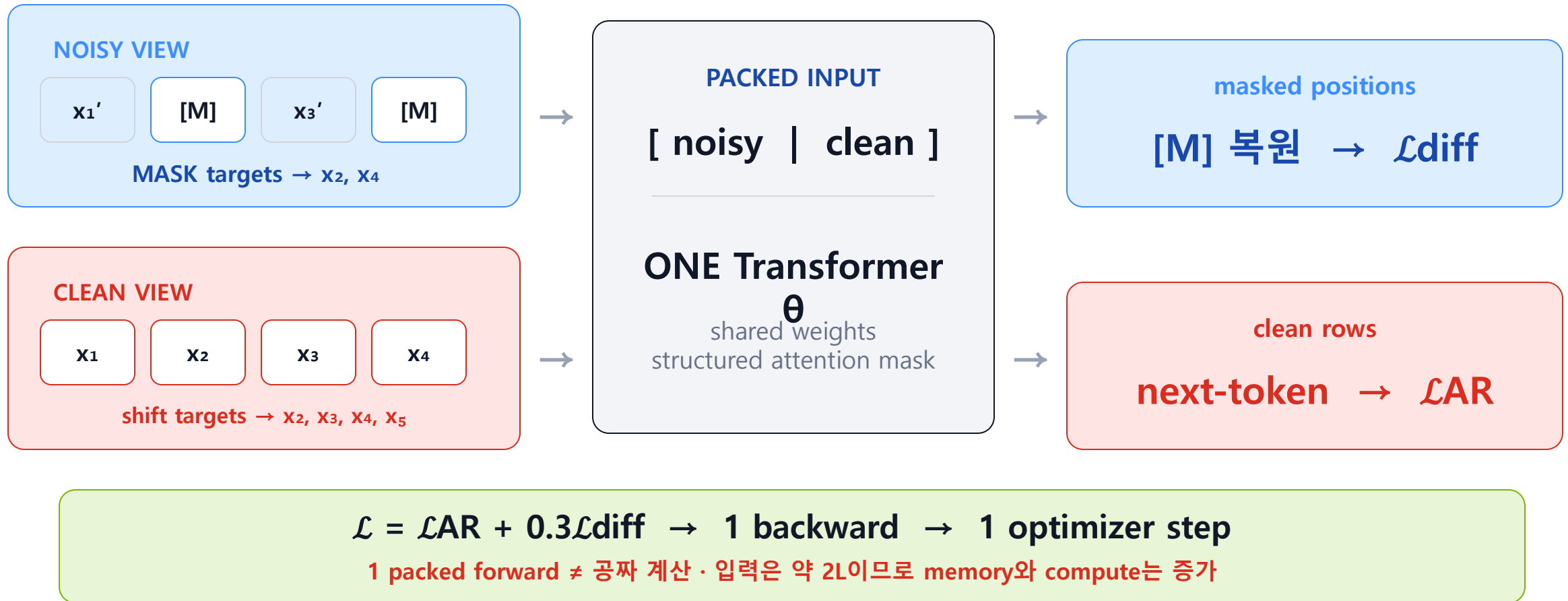
Clean stream

- strictly causal
- 미래 label leakage 차단
- AR loss 계산

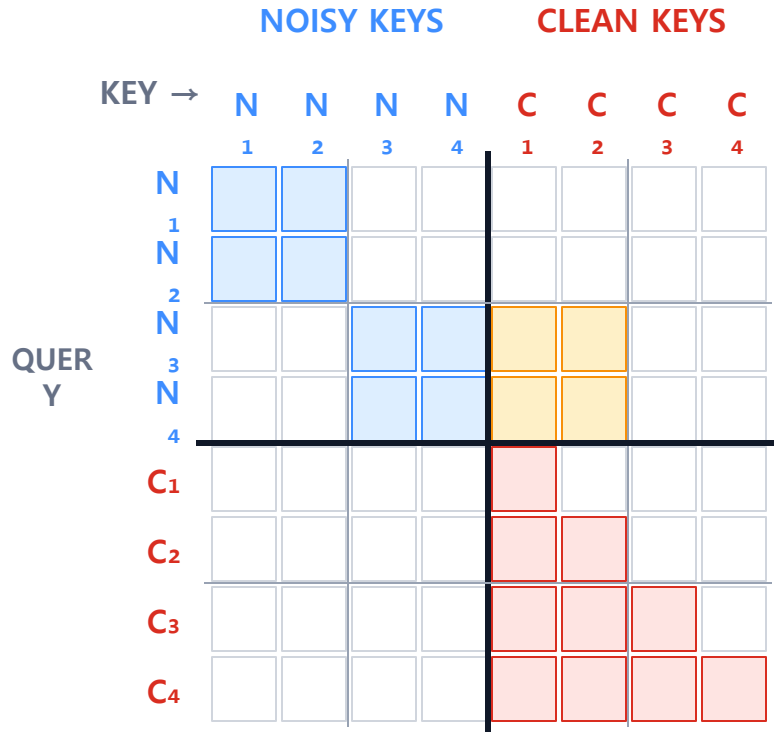
같은 forward-backward에서 두 objective가 동일한 θ 를 업데이트

한 문장 → 두 view → 한 Transformer → 두 loss

같은 문장 $x = [x_1 \ x_2 \ x_3 \ x_4]$



Attention mask 하나가 두 학습 규칙을 만든다



색이 칠해진 key만 볼 수 있다

- **Noisy → Noisy**
현재 block 내부 bidirectional
이전 noisy block은 차단
- **Noisy → Clean prefix**
현재 block b가 이전의 깨끗한
정답 문맥 $x_{<b}$ 를 추가로 참조
- **Clean → Clean**
strictly causal
미래 token과 noisy stream은 차단

Noisy rows → MASK 위치의 $\mathcal{L}_{diff}$

Clean rows → one-token-shift $\mathcal{L}_{AR}$

현재 noisy block = 현재 block(bidirectional) + 이전 clean prefix · 이전 noisy block은 차단

AR로 시작하고 SFT까지 두 loss를 유지한다

STAGE 1

Pure AR

1T tokens

$\alpha = 0$

STAGE 2

Joint AR + Diff.

300B tokens

$\alpha = 0.3$

SFT

Joint instruction tuning

45B tokens

answer-only loss

Joint instruction tuning = prompt는 깨끗하게 유지 · answer 영역만 masking/loss · $\mathcal{L}_{AR} + 0.3\mathcal{L}_{diff}$

256

학습에 사용한 H100 GPU 수

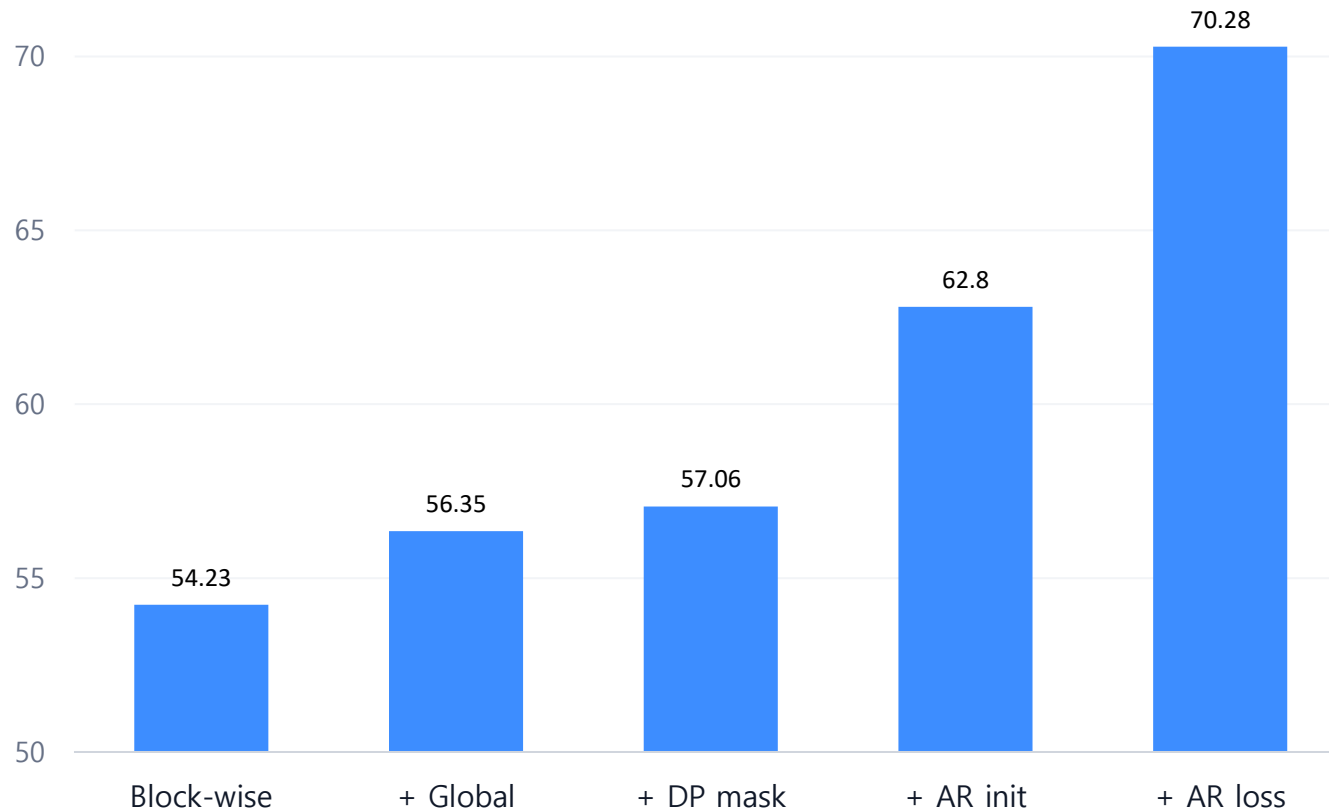
4,096

base 학습의 최대 token 길이

16K

instruction SFT 최대 token 길이

AR prior는 '좌→우 언어 구조를 이미 아는 가중치'다



AR prior $\neq$ Bayesian prior
AR prior $\neq$ 별도 네트워크

+5.74%p

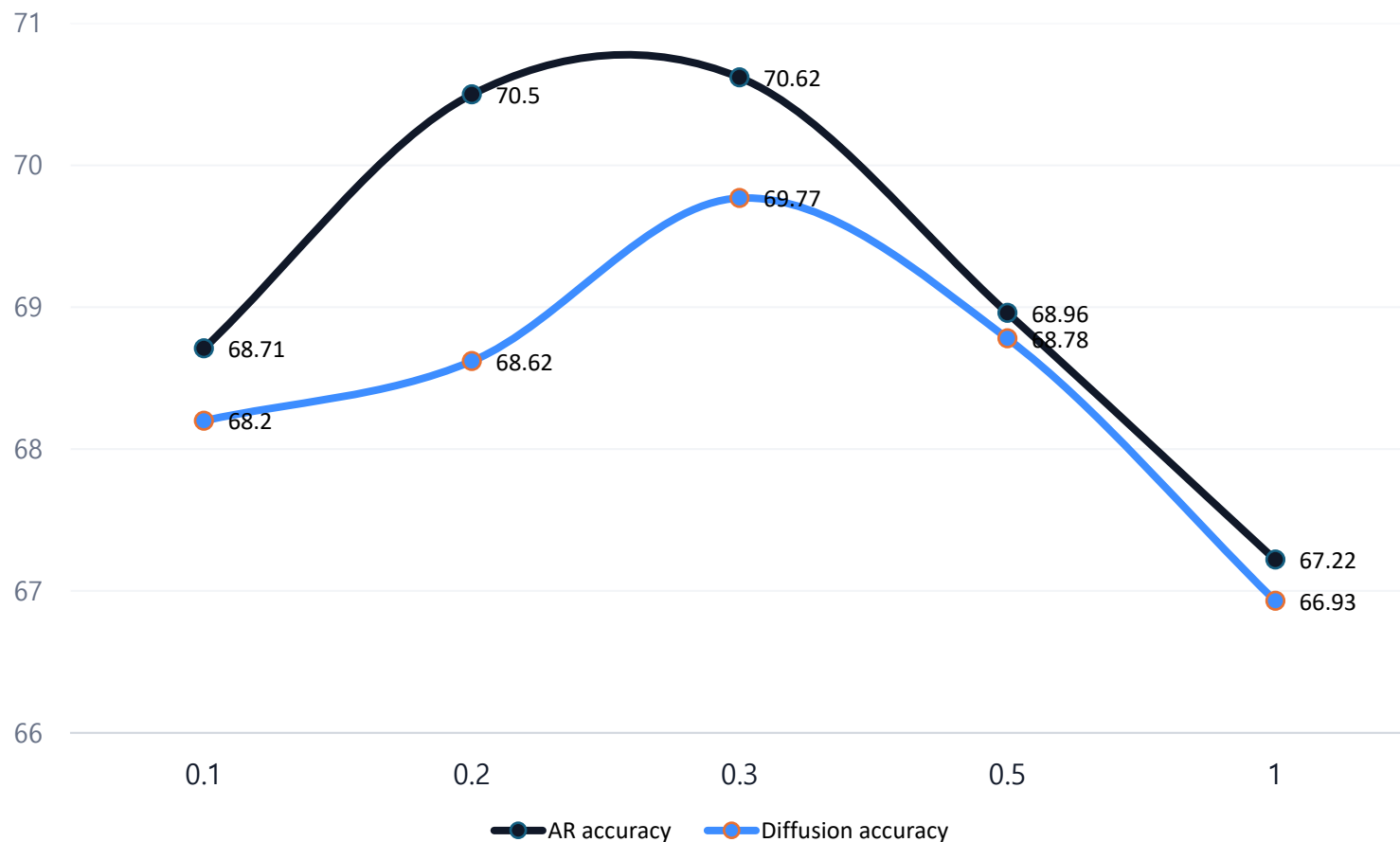
Stage 1 AR 초기화 효과

+7.48%p

Stage 2에서 AR loss 유지 효과

강한 AR 가중치에서 시작하고 AR loss를 계속 주면, diffusion 복원도 문법적 좌→우 궤적에 고정된다

$\alpha=0.3$ 에서 두 모드가 함께 좋아졌다



0.3

공동 최적점

두 모드가 서로 희생하지 않고 함께 peak

+0.43%p

Instruct AR 평균

diffusion loss 추가 후 오히려 소폭 상승

Dual-stream은 훈련 장치, tri-mode는 추론 경로다



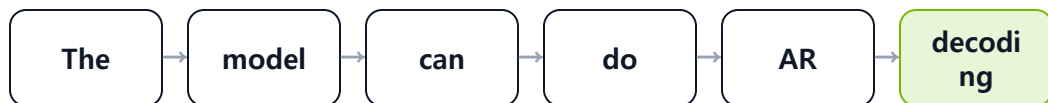
두 loss는 checkpoint 하나를 만들고, tri-mode는 그 checkpoint의 세 가지 사용법이다
추론에는 정답 clean copy · loss · backward가 없다 · diffusion의 clean prefix는 이미 생성된 과거 block

추론 Mode 1 — AR-only decoding

학습은 joint · 이 inference pass에서는 causal AR만 사용

$$p_{\theta}(x_i \mid x_{< i})$$

causal attention · 1 token / forward



언제 쓰나

- 높은 동시 사용자 수
- 큰 batch로 GPU가 이미 compute-bound
- 기존 AR serving pipeline 유지 필요
- 가장 보수적인 품질 기준이 중요

Mode 1은 inference에서 AR만 실행하지만, checkpoint 자체는 AR+diffusion으로 공동 학습됐다

추론 Mode 2 — diffusion-only decoding



이 모드의 경계

- AR verifier 없음
- confidence threshold 또는 learned sampler 선택 가능
- 이전 완료 block은 clean prefix/KV cache

$$\text{TPF} = \text{block size} / \text{denoising forward 횟수}$$

sampler는 '무엇'보다 '언제'를 학습한다

BACKBONE

이 MASK에 무엇이 들어가는가?

나는 오늘 회사에 갔다

confidence: 0.91 / 0.93 / 0.89 / 0.92

SAMPLER

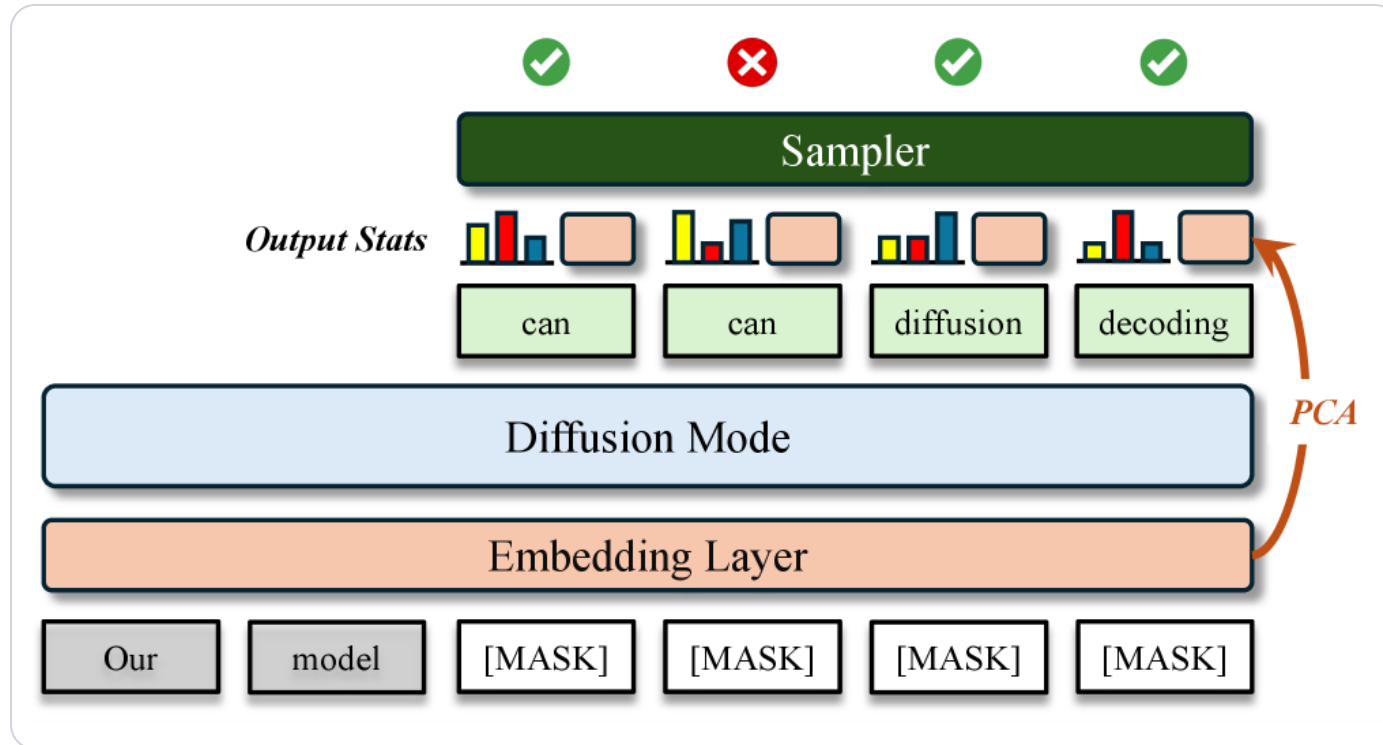
지금 확정해도 끝까지 유지되는가?

✓ ✓ ✗ ✓

'회사에'는 확률이 높지만
문맥이 채워지면 '학교에'로 바뀔 수 있음

높은 confidence ≠ 최종 안정성 | 여러 위치를 동시에 commit하면 서로의 조건부 분포가 바뀐다

Sampler는 frozen 8B 위에 붙는 별도 4-layer classifier다



4 layers

bidirectional Transformer · $h=384$

4.8M

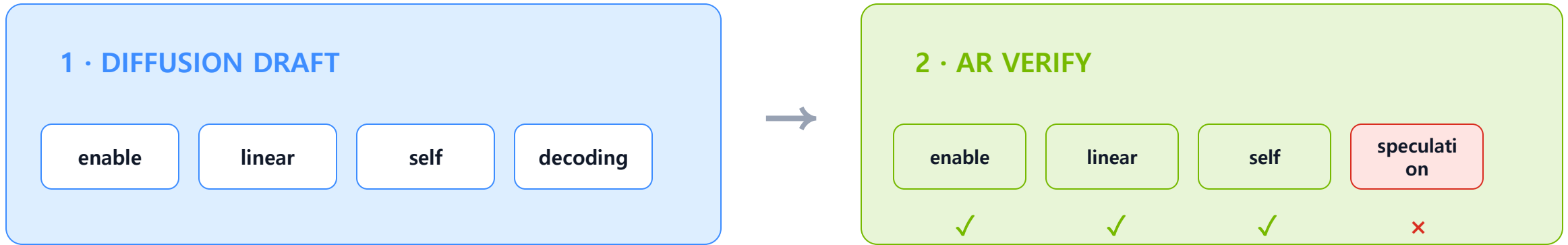
8B backbone의 약 0.06%

20M

denoising trajectories로 학습

본체가 token을 예측하고, sampler는 위치별 $p(\text{commit now})$ 를 내는 binary acceptance classifier

추론 Mode 3 — diffusion draft + AR verify



longest matching prefix

`enable linear self`

+ first rejected position의 AR token

`speculation`

같은 θ 가 두 번 forward · 별도 draft model 없음 · 1~k+1 tokens commit

왜 Quadratic TPF가 더 높은가? 분모가 1 forward이기 때문

LINEAR SELF-SPECULATION

Forward 1
Diffusion draft
t



Forward 2
AR verify

TPF = 확정 토큰 수 ÷ 2 forwards

reported Avg TPF

5.99

규칙적인 block draft + causal verify
입력 규모는 speculative width k에 대체로 선형

QUADRATIC SELF-SPECULATION

Forward 1
현재 draft 검증 + 다음 draft 후보 준비

TPF = 확정 토큰 수 ÷ 1 forward

reported Avg TPF

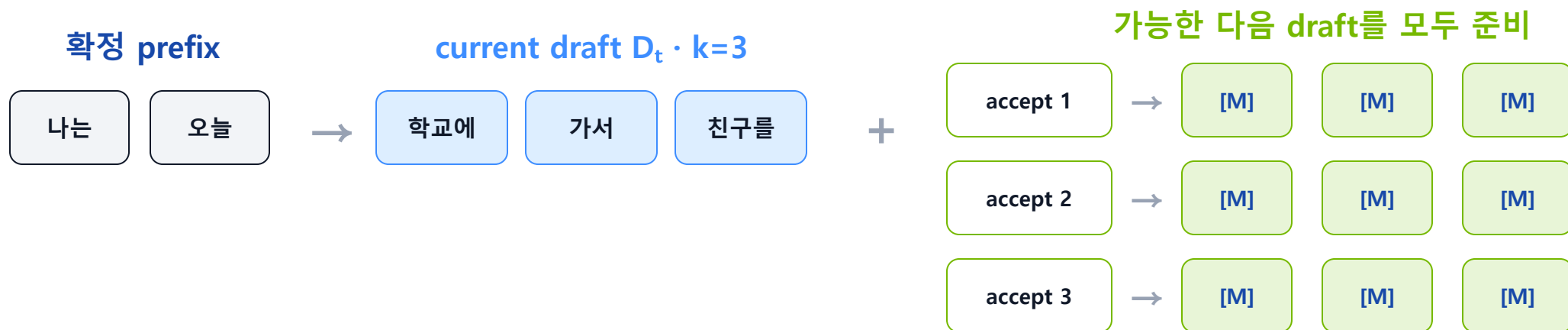
6.38

어디서 reject될지 몰라 모든 다음 후보를 준비
 k^2 MASK + structured attention

TPF는 forward '호출 횟수'만 센다. Quadratic의 한 forward가 더 무거워도 분모에는 똑같이 1로 들어간다.

Quadratic 1-pass: 현재 draft 검증과 다음 draft 준비를 겹친다

전제: cycle t 가 시작될 때는 이전 cycle이 만든 current draft D_t 가 이미 존재한다



A · 같은 forward의 causal 경로

current draft를 왼쪽부터 검증 → 첫 mismatch j^* 결정

B · 같은 forward의 diffusion 경로

accept 길이 $1...k$ 각각에 대응하는 next draft를 병렬 계산

검증 결과 j^* 에 맞는 branch 하나만 선택해 D_{t+1} 로 넘김 → 1 pass/iteration, 대신 k^2 MASK 비용

LoRA는 draft-verify 정렬, Quadratic은 kernel 병목

왜 LoRA가 나오나?

Diffusion draft
er
+ o_proj LoRA

Frozen AR veri
fier
분포에 정렬

4.52 → 5.99 TPF

+32.5% · 정확도 62.88 → 62.81
36M parameters · backbone의 약 0.4%

왜 Quadratic이 실제로는 느린가?

Linear

규칙적인 draft pass
+ causal verify pass

Quadratic

k^2 MASK + 비정형
structured attention

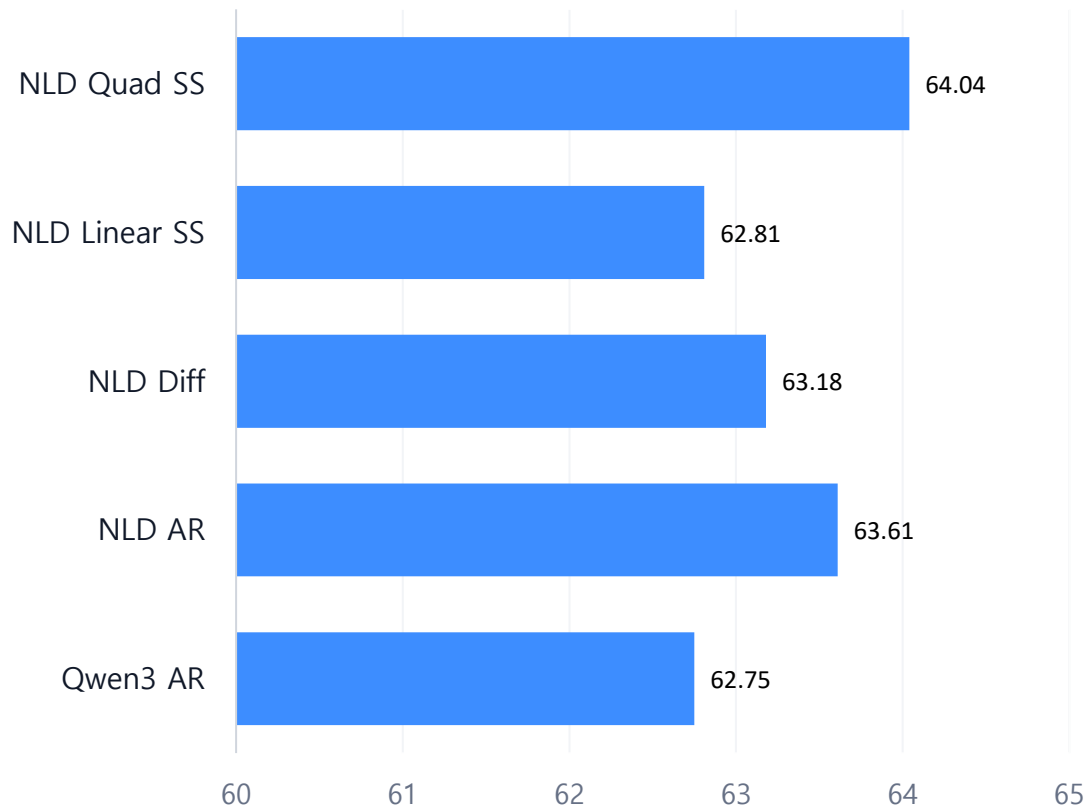
전용 mask 때문에 FlexAttention 사용
현재 kernel 최적화가 덜 되어 real-device
성능이 낮음

※ 논문이 지적한 것은 FlashAttention 부담이 아니라 FlexAttention의 미성숙한 전용 kernel

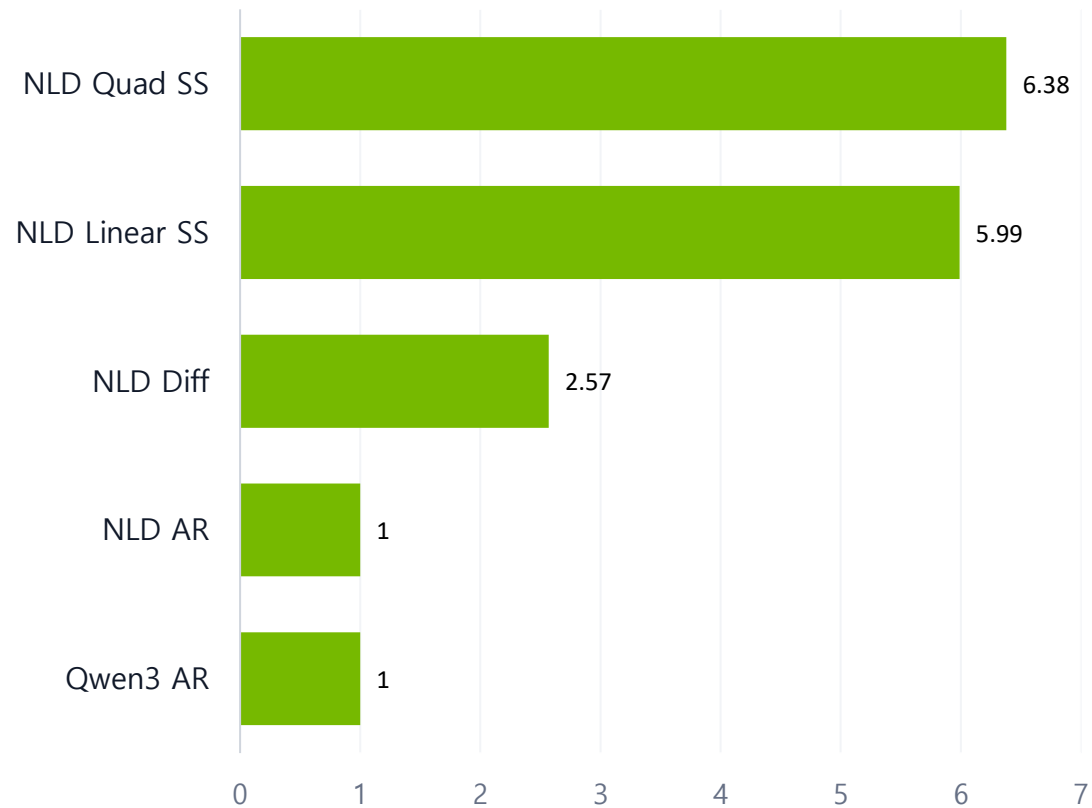
LoRA는 새 mode가 아니라 drafter의 말투를 verifier에 맞추는 작은 post-training adapter

한 checkpoint에서 mode별 정확도와 TPF를 바꾼다

평균 정확도 (%)

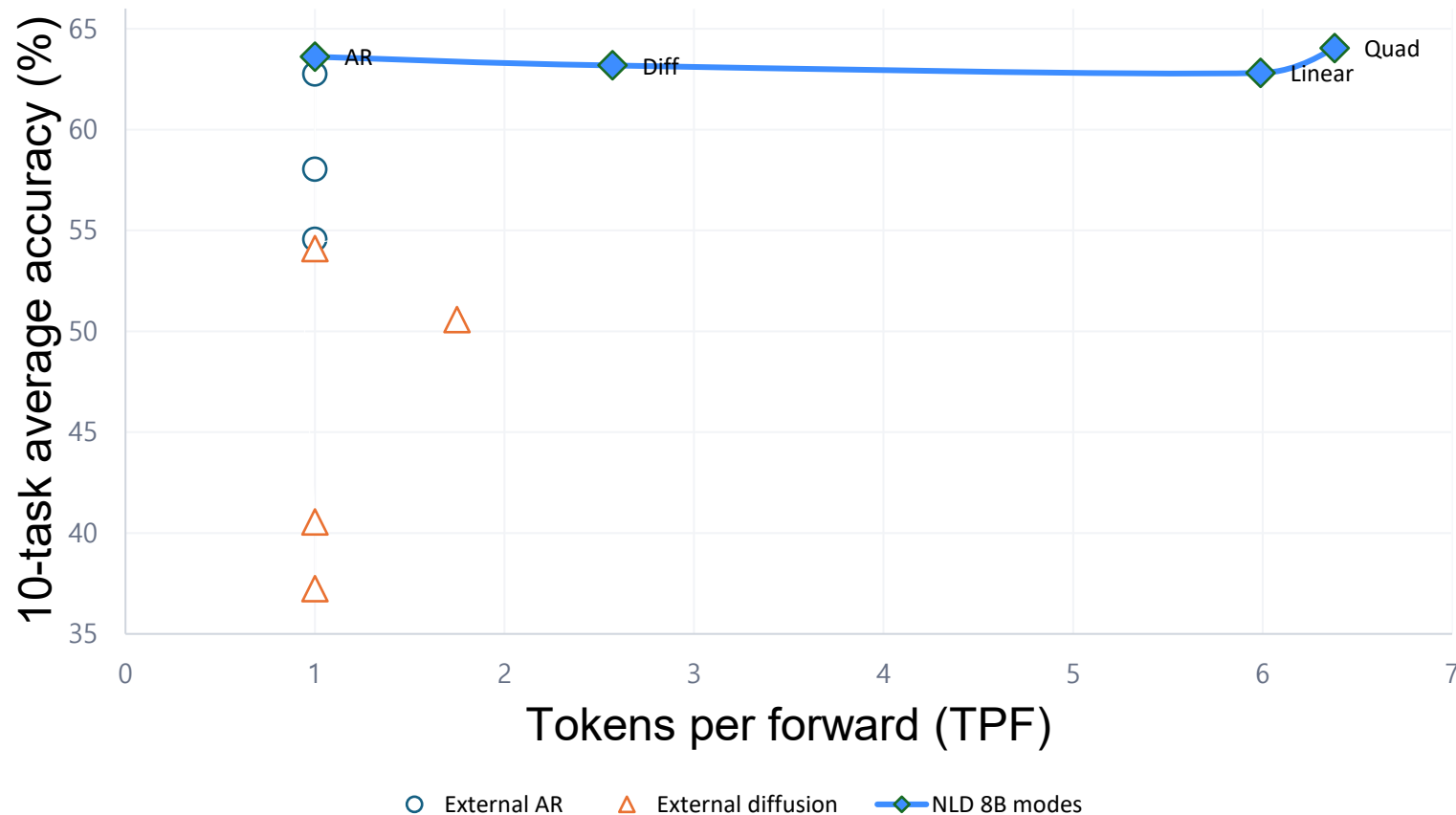


Tokens per forward



동일한 10개 benchmark 평균 · non-thinking · TPF는 wall-clock tok/s와 다름

다른 7-8B LM과 비교해도 Pareto frontier



NLD-8B · 같은 checkpoint

	t	Acc.	TPF
AR		63.61	1.00
Diffusion		63.18	2.57
Linear SS		62.81	5.99
Quadratic S		64.04	6.38

기존 diffusion 전용 모델보다
정확도와 병렬성을 동시에 확보

10 tasks · non-thinking · 외부 diffusion baseline은 각 공식 pipeline 사용 · TPF ≠ tok/s

Concurrency=1의 실제 GB200 속도: measured와 projected를 구분

256

NLD AR

tok/s · baseline · 1 token/
step

851

Linear SS

tok/s · measured · 3.32×
vs AR

1015

CUDA optimized

tok/s · measured · 3.97×
vs AR

354

Eagle3

tok/s · measured · Linear
SS가 약 2.4×

왜 TPF 5.99가 5.99× tok/s가 아닌가?

실제 속도에는 draft-verify 두 pass, KV cache 갱신, memory 이동, reject된 token 계산, kernel 효율이 모두 포함된다.

c=1은 동시에 처리하는 sequence가 하나인 저지연·단일 사용자 조건

저동시성은 사용자 속도, 고동시성은 시스템 처리량

LOW CONCURRENCY · 예: 요청 1개

8B weight read
GPU가 덜 참

AR: 1 token
SS: 여러 token 승인

weight를 한 번 읽을 때 더 많은 위치를 계산해
한 사용자의 tok/s를 크게 올린다

최적 선택: Linear self-speculation

HIGH CONCURRENCY · 많은 동시 요청

8B weight read
큰 AR batch

요청 N개의
next token을
병렬 처리

AR batch만으로 GPU가 compute-bound에 가까워
지고
SS의 draft-verify-reject 연산이 추가 비용이 된다

최적 선택: AR mode가 system throughput에
유리

Per-user throughput = 한 사용자 속도 | System throughput = 서버 전체 사용자 속도의 합

Linear SS는 실제 decoder, Diffusion SOL은 offline oracle

LINEAR SELF-SPECULATION

성격	실제 배포 가능한 decoding
구성	Diffusion draft + AR verify
비용	cycle마다 2 forwards
채택	첫 mismatch 전의 연속 prefix
정합 기준	AR verifier의 출력

DIFFUSION SOL

성격	실제 사용 불가 · offline ceiling
구성	Diffusion only · AR verifier 없음
비용	accept step 1 forward + cache refresh
채택	비연속 위치도 가능한 safe subset
정합 기준	serial diffusion의 최종 출력 t

둘은 같은 runtime 알고리즘의 속도 대결이 아니라, 같은 checkpoint가 가진 병렬 decoding 잠재력의 비교

SOL oracle: serial target $t \rightarrow$ 미래를 아는 simulation $\rightarrow$ safe subset



각 safety check가 다시 decoding을 simulation · 최대 5,000 extra forwards/block $\rightarrow$ 제품용 sampler가 아니라 offline upper bound

왜 SOL을 만드는가? backbone 병목과 sampler 병목을 분리한다

Diffusion backbone의 한 forward



여러 위치의 top-1 후보 + confidence



Sampler가 지금 확정할 위치 선택

모델이 맞힌 양과 sampler가 골라낸 양은 서로 다른 문제

상한이 현재와 비슷하다면

현재 sampler $\approx$ SOL ceiling

- backbone 자체가 동시에 맞힐 수 있는 토큰이 적다
- sampler만 개선해도 얻을 이득이 작다

상한이 훨씬 높다면 · NLD 결과

confidence sampling $\approx$ 3 TPF $\ll$ SOL 7.60 acceptance

- backbone은 이미 여러 위치를 맞히고 있다
- selection policy가 병렬성을 놓치고 있다

SOL은 새 decoder가 아니라 '더 좋은 sampler를 연구할 가치와 목표점'을 수치로 만드는 진단 도구

SOL 수치 읽기: acceptance는 비슷하지만 real TPF는 76.5% 차이

Acceptance per step

7.60

Diffusion SOL

oracle safe subset

6.82

Linear SS

SOL보다 약 10.3% 낮음

Real tokens per forward

6.02

Diffusion SOL

cache refresh 포함 ceiling

3.41

Linear SS

2 forwards/cycle

+76.5% real TPF headroom

GB200 · concurrency 1 · tok/s

851

Linear SS · measured

1015

CUDA optimized · measured

1471

SOL · projected ceiling

1471 tok/s는 oracle을 온라인에서 실행한 측정치가 아니라 SOL acceptance를 현재 infrastructure에 대입한 추정치

이 논문의 강점은 '통합'과 '전환 가능성'

- | | | |
|----|---------------------|---|
| 01 | 한 checkpoint, 세 운영점 | 모델 교체 없이 concurrency에 맞춰 decoding mode 전환 |
| 02 | joint training을 실증 | AR prior가 diffusion 정확도를 높인다는 ablation 제공 |
| 03 | MTP와 시스템 단위 비교 | TPF가 아닌 실제 GPU throughput과 acceptance length 측정 |
| 04 | 상한과 현재 구현을 분리 | SOL 분석으로 sampler 연구의 남은 공간을 정량화 |

다만 속도 주장은 조건과 구현에 민감하다

- | | | |
|---|--------------------------------|---|
| 1 | Benchmark comparability | AR baseline은 공통 프레임워크, diffusion baseline은 공식 파이프라인 사용 → 완전 동일 조건은 아님 |
| 2 | Sampler generalization | 20M trajectory가 math·code·STEM·chat 중심 → 다른 도메인과 긴 문맥에서 재검증 필요 |
| 3 | Oracle vs deployable | SOL의 safe subset 탐색은 실제 inference에서 쓸 수 없는 상한 |
| 4 | System dependence | quadratic SS가 TPF는 높아도 kernel 미성숙으로 실제 속도는 낮음 |

Reviewer's take: '6× TPF'와 '4× throughput'은 같은 주장이 아니며, 배포 조건을 함께 읽어야 한다.

한 checkpoint를 학습하고, inference 경로를 상황에 맞게 고른다.

01

Training

$\mathcal{L}_{AR} + 0.3\mathcal{L}_{diff}$ 로 하나의 tri-mode backbone

02

Inference

Mode 1 AR · Mode 2 Diff ·
Mode 3 Self-spec

03

Deployment

낮은 동시성: self-spec · 높은
동시성: AR

04

Open problem

SOL에 가까운 non-prefix sampler

토론 질문

① sampler를 end-to-end로 학습할 수 있을까? ② 문단 단위 병렬성까지 확장할 수 있을까?